

Project 1

Team :

Ashwin

Vishnu Surya Teja (Lead)

Anjaneyulu

Project Overview AetherNexus is an autonomous, self-healing, multi-region cloud infrastructure simulator. It serves as a digital proving ground to validate how Large Language Models (LLMs) can autonomously monitor server telemetry, detect critical infrastructure anomalies, and execute programmatic mitigation routines (e.g., memory cache flushing, dynamic traffic redistribution) in real-time with zero human intervention.

Technology Stack

- **Backend Simulation & Gateway:** Node.js, Express, TypeScript
- **Database Layer:** MongoDB Atlas
- **AI Orchestration:** Llama-3.3-70b-versatile (via Groq API), Model Context Protocol (MCP) SDK
- **Frontend Dashboard:** React, Vite
- **3D Rendering Engine:** Three.js, React Three Fiber (R3F)
- **Real-Time Data Transport:** Socket.io (WebSockets)

Project Scope

- **In-Scope:**
 - Engineering a backend chaos engine to simulate multi-region telemetry (US-East, EU-West, AP-South).
 - Integrating an autonomous AI agent to poll telemetry metrics on a strict 30-second temporal loop.
 - Registering deterministic MCP tools to execute infrastructure mitigation.
 - Developing a real-time, WebGL-powered 3D dashboard for state visualization and incident logging.
- **Out-of-Scope:**
 - Integration with real-world production cloud billing or instances (AWS/GCP/Azure).
 - Ingestion and routing of actual, external client web traffic.
 - Mobile or responsive application development for the dashboard.

Project is structured across three distinct operational Parts: Backend(the Simulation Engine), Middle layer(the Autonomous AI Core), and Frontend(the Matrix Command Center).

Domain 1: Multi-Region Chaos Engine (Backend)

This tier serves as the simulated infrastructure environment, generating dynamic telemetry data to train and test the AI Core.

- **API Gateway & Load Balancer:** Acts as the network ingress point. It processes live routing tables from MongoDB Atlas and distributes simulated incoming traffic proportionally across three geographical clusters (US-East, EU-West, AP-South).
- **Chaos Server:** Generates fluctuating operational metrics (CPU spikes, memory limits). It exposes programmatic endpoints (e.g., `POST /api/chaos/spike-cpu`) allowing administrators to manually inject faults and test the system's resilience.
- **Socket.io Egress Broadcaster:** A WebSocket tunnel that streams localized server state changes directly to connected frontend clients in real-time.

Domain 2: Autonomous AI Core (Intelligence)

This tier is the deterministic AI guardian. It relies on the Model Context Protocol (MCP) to interact programmatically with Domain 1.

- **The Evaluation Clock:** A Node.js setInterval loop that strictly executes an AI evaluation cycle every 30 seconds.
- **LLM Engine & Key Pooling:** Utilizes the Llama-3.3-70b-versatile model (via Groq API). To ensure high availability, it implements a rotational API Key Pool; if a 429 Rate Limit error occurs, it autonomously switches keys and retries the process without crashing.
- **The Rulebook (skills.md):** Serves as the system prompt. It enforces strict behavioral guidelines, telling the AI how to interpret CPU thresholds and explicitly dictating which MCP tools to invoke during specific anomalies.
- **The MCP Toolbelt:** Custom programmatic tools registered within the AI context, including:
 - `fetchLiveInfrastructureMetrics`: Retrieves live arrays of system health.
 - `traceRepositoryCommitHistory`: Scans git logs for recent breaking changes.
 - `executeClusterCacheFlush`: Restores overloaded servers to baseline stability.
 - `executeLoadBalancing`: Modifies routing weights to divert traffic from failing nodes.
 - `requestHumanOverrideClearance`: Pauses automated execution and pings administrators for hardware reboots.

Domain 3: 3D Matrix Telemetry Grid (Frontend)

This tier provides enterprise-grade visualization of the infrastructure and the AI's decision ledger.

- **The 3D Viewport:** Built with Three.js and React Three Fiber (R3F). It renders glowing, animated 3D monoliths representing the server clusters.
- **State Binding:** The frontend actively listens to the Socket.io stream. When the AI detects an anomaly, the specific 3D node instantly updates its emissive material to pulse red; when mitigated, it transitions back to green.
- **The Action Ledger:** An interactive UI panel that decodes the AI's internal reasoning loop, translating complex technical mitigations into clear, human-readable audit logs (e.g., "Mitigation Complete: Cache flush executed on euWestCluster").

The Operational Data Flow Pipeline

The AetherNexus architecture operates in a continuous, multi-domain loop. Below is the step-by-step data flow demonstrating how the system transitions from passive monitoring to autonomous incident mitigation.

Phase 1: Normal Operation (Passive Monitoring)

- **Step 1 (Generation):** The Chaos Engine (Backend) generates nominal telemetry metrics (e.g., CPU load at ~35%).
- **Step 2 (Polling):** The AI Core (Llama-3.3) executes `fetchLiveInfrastructureMetrics` via the MCP tunnel every 30 seconds.
- **Step 3 (Evaluation):** The AI determines the state is stable against its `skills.md` thresholds.
- **Step 4 (Visualization):** The AI broadcasts a `NOMINAL_GREEN` status via Socket.io. The 3D Matrix grid renders all nodes as stable, green monoliths.

Phase 2: Incident Detection (The Chaos Event)

- **Step 1 (Injection):** An administrator triggers a fault via the `POST /api/chaos/spike-cpu` endpoint on the EU-West cluster.
- **Step 2 (Propagation):** The Backend simulates a critical load spike (CPU jumping to 98%).
- **Step 3 (Detection):** On the next 30-second polling cycle, the AI Core ingests the critical metrics and identifies a threshold breach.
- **Step 4 (Alerting):** The AI broadcasts a `CRITICAL_RED` status. The frontend UI instantly transitions the EU-West 3D node to red, scaling its geometry to visually represent stress.

Phase 3: Autonomous Mitigation (Self-Healing)

- **Step 1 (Decision):** The AI Core cross-references the anomaly with its `skills.md` directives and decides to execute a cache flush.
- **Step 2 (Execution):** The AI triggers the `executeClusterCacheFlush` MCP tool targeting the EU-West cluster.

- **Step 3 (Recovery):** The Backend receives the command, clears simulated memory buffers, and artificially lowers the CPU load back to a 45% baseline.
- **Step 4 (Confirmation & Healing):** On the next cycle, the AI Core confirms the metrics have stabilized. It logs the successful mitigation ("Cache flush executed") and broadcasts a recovery event. The 3D Matrix grid transitions the EU-West node back to green.

Project Plan & Team Contributions

AetherNexus was built through a collaborative, phased integration strategy. While the project is unified, the development was structured into three distinct ownership tracks.

The Team & Skills Acquired

- **Ashwin (Backend & Chaos Engineering)**
 - *Focus:* Designed the Domain 1 Simulator and Express APIs.
 - *Skills Learned:* Node.js asynchronous event loops, probabilistic simulation logic, JSON schema modeling, and memory management using circular data buffers.
 - *Challenges Overcome:* Tuning the Chaos Generator to inject faults frequently enough for observation without completely overwhelming the server thread.
- **Vishnu Surya Teja (AI Core & MCP Integration)**
 - *Focus:* Designed the Domain 2 Intelligence Layer and Groq API implementation.
 - *Skills Learned:* Model Context Protocol (MCP) tool definition, Large Language Model prompt engineering, API Key pooling/rotation strategies, and enforcing deterministic JSON output.
 - *Challenges Overcome:* Eliminating LLM hallucinated conversational text by writing strict, parseable data rules in the `skills.md` system prompt.
- **Anjaneyulu (Frontend Visualization)**
 - *Focus:* Designed the Domain 3 Command Center and 3D environment.
 - *Skills Learned:* Three.js scene composition (React Three Fiber), dynamic emissive material animation, Socket.io real-time state binding, and absolute UI overlays.
 - *Challenges Overcome:* Synchronizing the high-speed WebGL render loop with React state updates to prevent re-render storms and browser memory crashes (WebGL Context Loss).

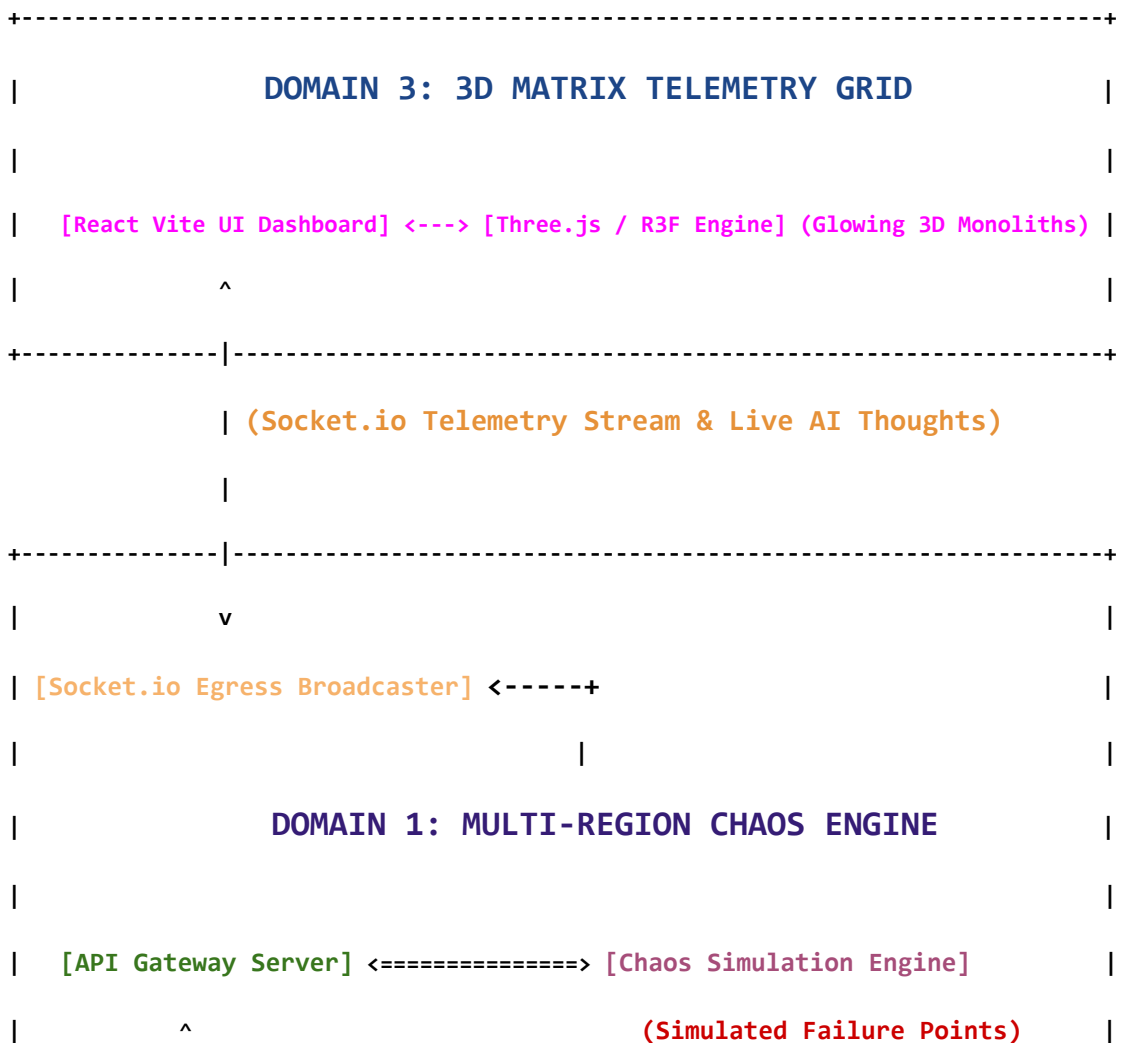
Integration Seams (How It Came Together)

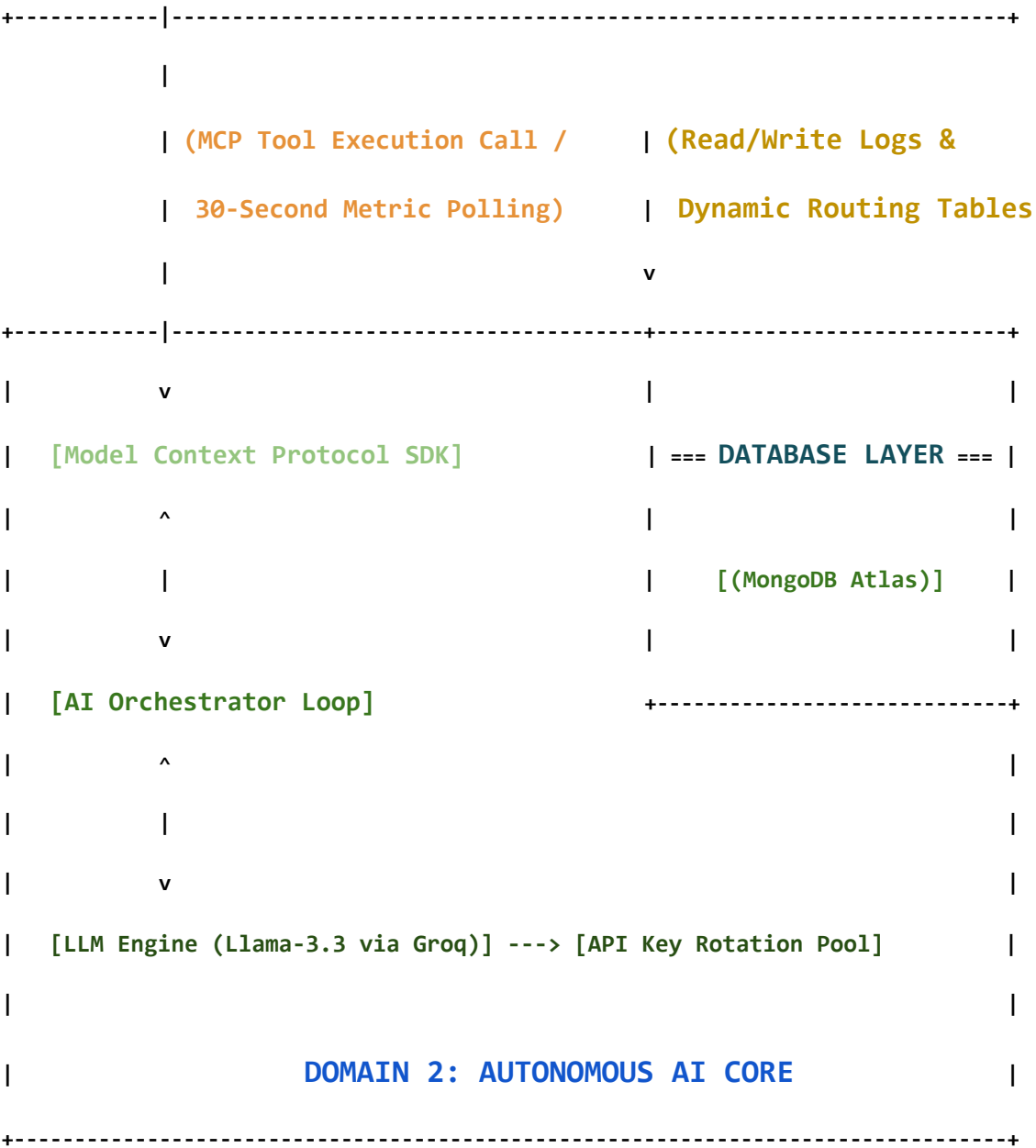
The success of the project relied on strict data contracts between the three phases:

1. **Phase 1** ➡ **Phase 2:** The Backend exposed the GET `/api/infrastructure/telemetry` endpoint. The AI Dev consumed this, ensuring the MCP

- fetchLiveInfrastructureMetrics tool knew exactly what JSON fields to expect (cpu, ram_gb, network_pps).
2. **Phase 2** ➡ **Phase 3:** The AI Dev defined strict string constants for their Socket.io broadcast (e.g., CRITICAL_RED, NOMINAL_GREEN). The Frontend Dev built their 3D state machine logic entirely around those specific string payloads.
 3. **The Human-in-the-Loop Handshake:** The requestHumanOverrideClearance event required everyone to collaborate: The AI paused its execution loop, the Backend handled the WebSocket escalation flag, and the Frontend mounted an interaction-blocking UI modal pending human approval.

AETHERNEXUS ARCHITECTURE





DOMAIN 2: AUTONOMOUS AI CORE